Assessment 1 - Adaptive Customer Service Harness

This is the CA for Advanced Architectures and Applications in Agentic AI, worth 40% of the module mark. This CA assesses Learning Outcomes 1 and 2 of the module:

1. Combine orchestration and autonomous decision-making patterns to develop self-directed and goal-oriented Agentic solutions.
2. Design and evaluate long-term memory strategies and retrieval techniques for agentic AI.

Objective

In this CA you will create an Agent harness that uses memory to learn from the environment and develop new skills. The objectives are broadly based on the learnings from the CoALA paper and Agent Skills.

Scenario

Develop a harness that will provide automated service desk support. A eval set is provided to develop a service agent's skills.

Each use case in the eval set consists of a customer query and a example response. For example:

Customer Query	**Model Response**
I need to cancel purchase {{Order Number}}	I understood that you need assistance with canceling your purchase with the order number {{Order Number}}. We apologize for any inconvenience this may have caused. To initiate the cancellation process, please follow these steps: 1. Sign in to your account: Visit our {{Online Company Portal Info}} and login using your credentials. 2. Access your order history: Once logged in, navigate to the "{{Online Order Interaction}}" or "{{Online Order Interaction}}" section.

3. Locate the purchase: Look for the order with the specific order number {{Order Number}}.
4. Cancel the purchase: Click on the "{{Online Order Interaction}}" option associated with the purchase.
5. Confirm the cancellation: Follow any further prompts or questions from the system to finalize the cancellation.

If you encounter any difficulties or have further questions, our dedicated team is available to assist you. You can reach us during {{Customer Support Hours}} at {{Customer Support Phone Number}} or through the Live Chat feature on our website {{Website URL}}. We appreciate your understanding and value your satisfaction.

Initially, your application will not have any knowledge of how it should answer the customer queries, it will just guess.

You're harness is then going to learn from sample queries. Each time it does not have a skill to answer a query, it will ask a 'supervisor' who will provide the example response from the training data. The agent then needs to ingest the response as a new skill.

In addition, during training the harness will evaluate the quality of the response against the example response, and curate a set of guidelines that direct the agent on how to respond to queries. The guidelines will be contained in a AGENT.md file, with a maximum of 10 guidelines, which will be improved after each training query (see CoALA procedural memory as a guide).

Once training is finished, the harness will use skills and guidelines it has created while learning from the training data.

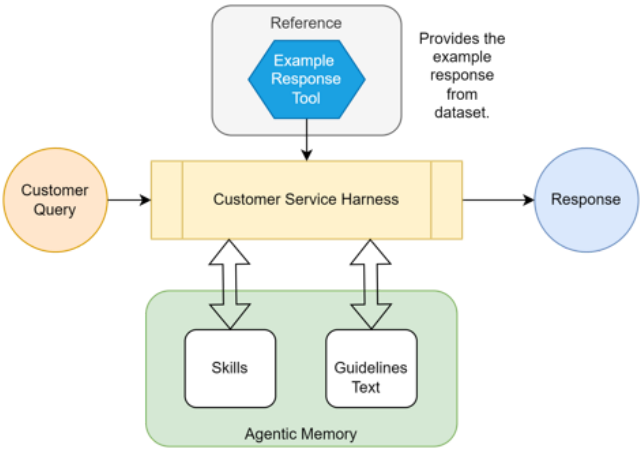
Design

Your system will process the customer queries, and where it does not have enough information to answer the question it will use a tool (you provide during training only) to get the example response from the training data.

The agent will respond with the benefit of the example response in its context (not exactly the same, but in the agents own words).

After the agent has responded your application will analyse the conversation to see if the agent needed help i.e. looked at the example response), and it will ingest the response as a skill to be used for similar queries in the future.

At the end of every query during the training cycle the harness will also compare the ‘style’ of the agent’s response to develop guidelines for how the agent should respond.



Guidelines should include where the agent gave the wrong kind of response (maybe offered detailed advice rather than ask for more information) as well as where the tone of the conversation could be improved. Use a Reflection/'LLM as a Judge' approach to develop the guidelines.

The guidelines should be stored in an AGENT.md file.

Testing and Evaluation

The outcome should be that the agent learns from the example responses. To assess this, additional examples can be sampled as a test set and scored using a typical ‘LLM-as-a-Judge’ evaluator process in LangSmith (note that this evaluation done in langsmith is different from the evaluation used to develop guidelines, which runs inside the harness during training).

The initial level of improvement can be assessed by evaluating with and without Agentic memory. Effectiveness of subsequent training runs can also assessed to see whether further

incremental (or decremental) improvement can be achieved. It is important the LLM-as-Judge evaluator provides a score, so incremental improvement can be assessed.

Approach

Your harness needs to support two different modes - training and non-training (for testing or final use).

In both cases the agent must select the relevant skill from the list of skills in the system prompt and use progressive disclosure to get the detailed instructions.

In training mode, if the agent does not have a relevant skill it should use a tool to get the example response. The example response then needs to be ingested as a new skill (that is general enough for similar future queries).

Also during training there must be another agent process which scores the response stylistically against the example response and create/update a list of (up to) 10 guidelines. These guidelines should be stored (in a md file e.g. AGENT.md) and added to the system prompt.

So during training there are two additional steps (skills creation and guideline improvement). You cam decide whether these two additional steps sit only within the harness (but not in the agent loop), or whether the main agent is directed to perform these additional steps during training inside the loop.

The final result should be an agent which is directed by skills and agent guidelines, all of which have been automatically generated by the harness during training (i.e. a self training/self adaptive) system.

Ingestion and Retrieval of Guidelines

The technique in the CoALA paper of a max number of succinct points is effective. Allow the agent to update this list when it thinks it will improve performance. This will prevent the list of guidelines growing endlessly. The guidelines can be added to the system prompt for every query.

Dataset

The dataset provided is a list of customer queries and model responses. There is also additional information to help classify the different responses.

The column headers are:

- id: simple row counter
- flags
- instruction: customer query

- category: Broad topic area - order, account, payment etc.
- intent: type of query (e.g. place order, cancel order)
- response: Model response

There are 26,873 rows.

You should take a small small from the dataset for development and testing. The full set is provided so you can choose your own samples.

The data has different categories, and for each category there is an ‘intent' sub category. You should select 2 or categories and sample a different intents to get a representative range of responses. Aim for around 20 to 30 examples for training. Then sample your test evalset in a similar way.

Technologies

You must use the technologies below for your implementation. This helps demonstrate a clear link between delivered material and learning outcomes and also ensure consistent marking.

- Use Langchain/Langgraph for the harness
- Use Agent Skills and markdown files for scaffold.

Suggestions

Here are some suggestions for developing the application.

- Setup Langsmith early to feed the queries to your app. Then use different splits as you progress. Start with a split of only one or two queries and generate new splits as increase your training and then testing. This can save a lot of time in the long run.
- Another advantage of recording experiments in Langsmith is you can replay and modify prompts in the playground, which can help speed up development and ‘debugging’ prompts.
- Ensure you can easily switch between training and test mode to assess progress.
- Consider using a paid API service. Free services like Groq can be made to work, but once you move into training and evaluation you may hit rate limits. This can start to slow down your development. Langchain does provide functions to prevent hitting rate limits, but only by slowing down calls made to the LLM. Unless you do very large runs during development the cost should be fairly low (you can track it on your providers portal). The use case is intended to be relatively simple to help limit costs. One good approach can be to use a free API while developing code, where you will normally only be running one or two queries, and switch to a paid API for training and testing.

Output

Submit the follow material to support your learning outcomes for the assessment:

- Write up and Reflection Report, in Word of pdf format, containing:
  - Introduction
  - Details of design - system diagram(s), description of architecture and process
  - Implementation (include code snippets and prompts to demonstrate key aspects)
  - Examples of skills and guidelines developed, and how they were defined.
  - Finding: Observations and test results (evals and traces with evidence of execution).
  - Reflection - effectiveness and suggested enhancements, what worked, challenges etc.
  - Conclusion
- Code (either Jupyter files, Python files or both are acceptable)
- Declaration cover sheet.

Please produce documentation without assistance of AI. This will be the primary means to assess your understanding of the learning outcomes. Code will be used as supporting evidence. You may be asked to perform a demonstration and interview to support your submission. Marks will only be awarded for content that can be verbally explained.

Rubric

Indicative marking scheme

	Marks (%)	**Excellent**	**Good**	**Poor**
Setup Application and test environment	20	App driven by datasets in LangSmith full control over splits for different train and test sets.	File driven train and test environment for application.	Hardcoded train and test/ no test.
Application learns and stores skills	30	Multiple skills available. Reliably pulls most relevant skill into context through progressive	Two or more skills available. Regularly pulls relevant skill into context.	Limited or unreliable retrieval of correct skill or skill not effective.

		disclosure. Available skills are relevant to topic.		
Evaluation and test results	30	Full evidence of results demonstrating training and test (on non train data), with relative improvements from training.	Evidence of training with improvements. Some test.	Limited evidence of working application or learning.
Report and documentation quality	20	Well structured report with clear description of design, implementation and testing, with in depth understanding and insightful reflection.	Well structured report with evidence of working application and understanding of core principles and some reflection.	Poorly structured report (or evidence of AI generation), little evidence of understanding related to LOs.